

Guía de arranque: Gemini CLI, GitHub y tu primera entrega

Instala Gemini en VS Code, prepara tu repositorio, entrena con el oráculo del cuaderno y entrega tu propio proyecto en la misión **M1 · El Despertar del DOM** — sin perder de vista la regla de oro de la asignatura.

 M1 · El Despertar del DOM Gemini CLI GitHub 60–90 min de entrenamiento

EN ESTA GUÍA

- 0 · La regla de oro
- 1 · Gemini en VS Code
- 2 · Preparar GitHub
- 3 · Entrenar con el oráculo
- 4 · Tu entrega de la M1
- 5 · El README
- 6 · Entregar en la Arena
-  Checklist final

0 ANTES DE EMPEZAR

La regla de oro

«Puedes usar IA para todo, pero todo lo que entregues lo tienes que poder defender línea a línea: la nota no es por escribir código, es por entenderlo.»

Usar Gemini (o Claude Code, o cualquier otra IA) no penaliza. Lo que sí cuenta para tu nota es que, tras entregar, puedas explicar **cualquier** línea de tu código en la defensa oral: por qué está ahí, qué alternativa descartaste, qué pasaría si la quitaras. La IA que corrige tu entrega genera además **3 preguntas de defensa** sobre tu código concreto — solo las ve el profesorado, y tu nota es **provisional** hasta que las respondas en persona.

Por eso esta guía no dice «pídele a la IA que te haga el ejercicio». Dice: pídele que **te lo explique mientras lo construye contigo**, revisa cada cambio antes de aceptarlo, y pruébalo tú mismo en el navegador en cada fase.

Ojo: dos cosas distintas se llaman «Misión 1».

DÓNDE	QUÉ ES	¿SE ENTREGA?
Cuaderno de la U1 «Misión 1 — El oráculo de los números»	Mini-proyecto guiado del Nivel 2, con código de partida idéntico para toda la clase. Es <i>entrenamiento</i> : el apartado 3 de esta guía.	No
Arena «M1 · El Despertar del DOM»	Misión <i>abierta</i> : una página interactiva en HTML + CSS + JS puro, con la idea que tú elijas. Es lo que se puntúa: el apartado 4.	Sí

El oráculo te sirve para aprender a trabajar con la IA fase a fase. Lo que subes a la Arena es **tuyo**.

1 HERRAMIENTAS

Integrar Gemini CLI en VS Code

1.1 · Requisitos

- VS Code instalado.
- Node.js 20 o superior (compruébalo con `node -v` en una terminal).
- Una cuenta de Google — la usarás una sola vez para generar una **API key** gratuita, no para iniciar sesión en el CLI.

1.2 · Instalar el CLI

Abre la terminal integrada de VS Code con `Ctrl` + ``` (en teclado español esa tecla suele ser la `ñ`; en Mac el atajo también lleva `Ctrl`, no `Cmd`). Si no te sale, el menú *Terminal* → *New Terminal* siempre funciona. Después:

```
npm install -g @google/gemini-cli
gemini --version
```

Si el último comando responde con un número de versión, está instalado.

1.3 · Autenticarse con una API key

No uses «Login with Google». El inicio de sesión por navegador del CLI no funciona ahora mismo de forma fiable. Usa una API key: se configura una vez y evita el problema por completo.

- 1 Ve a aistudio.google.com/apikey (<https://aistudio.google.com/apikey>) con tu cuenta de Google y pulsa «**Create API key**». Es gratis, no pide tarjeta.
- 2 Copia la clave generada (empieza por `AIza...`).
- 3 Guárdala como variable de entorno para que el CLI la use siempre, sin tener que repetirla. En macOS/Linux, añade esta línea al final de tu `~/.zshrc` (o `~/.bashrc` si usas bash):

```
export GEMINI_API_KEY="tu-clave-aquí"
```

y aplica el cambio con `source ~/.zshrc` (o abre una terminal nueva). En Windows (PowerShell):

```
setx GEMINI_API_KEY "tu-clave-aquí"
```

y abre una terminal nueva para que se aplique.

- 4 Comprueba que la variable está disponible: `echo $GEMINI_API_KEY` en macOS/Linux, o `echo $env:GEMINI_API_KEY` en PowerShell, debe mostrar tu clave.
- 5 En la terminal integrada de VS Code, escribe `gemini` y pulsa `Enter`. Con la variable ya definida, el CLI la detecta automáticamente y arranca sin pedir ningún login por navegador.

Trátala como una contraseña. Vive solo en tu variable de entorno local: no la escribas en el código, ni en el `README.md`, ni en un `GEMINI.md`, ni en ningún archivo que vayas a comitear — **tu repositorio va a ser público**. Y si alguna vez se te cuela en un commit, no basta con borrarla en el commit siguiente: queda en el historial para siempre, así que **revócala** en AI Studio y genera otra.

1.4 · Conectarlo de verdad al editor

Con el CLI abierto, ejecuta el comando `/ide install`. Esto instala la extensión complementaria de Gemini para VS Code, que te da **diff nativo** (los cambios que propone Gemini se ven como un diff nativo, no como texto plano) y comparte automáticamente el contexto del archivo que tienes abierto.

Alternativa: si prefieres, busca «Gemini CLI Companion» en el panel de Extensiones de VS Code (icono de piezas de puzzle en la barra lateral, o `Ctrl + Shift + X`) e instálala directamente — hace lo mismo.

1.5 • Comandos que vas a usar

COMANDO	QUÉ HACE
<code>@app.js</code>	Incluye ese archivo en el contexto de tu siguiente mensaje.
<code>@oraculo/</code>	Igual, pero con toda una carpeta.
<code>/ide install</code>	Instala y engancha la extensión de VS Code (apartado 1.4).
<code>/clear</code>	Limpia la conversación (útil al cambiar de fase del ejercicio).
<code>/help</code>	Lista todos los comandos disponibles.
<code>/quit</code> o <code>Ctrl + C</code> x2	Sale del CLI.

No actives el modo de aceptación automática («YOLO»). Por defecto, Gemini te pide confirmación antes de escribir o modificar cada archivo — déjalo así. Existe un modo que acepta todos los cambios sin preguntar; para aprender (y para poder defender tu código después) es justo lo contrario de lo que te conviene. Revisa siempre el diff antes de aceptar.

1.6 • Opcional: contexto del proyecto

Igual que este repositorio tiene un `CLAUDE.md` para darle contexto a Claude Code, Gemini lee un archivo `GEMINI.md` en la raíz del proyecto si existe. Para un ejercicio de una sesión no hace falta, pero si luego trabajas en un proyecto más grande, ahí puedes explicarle convenciones, estructura, etc. Recuerda que ese archivo **se sube al repositorio**: contexto sí, credenciales nunca.

2 HERRAMIENTAS

Preparar GitHub

2.1 • Cuenta de GitHub

Si no tienes cuenta, créala en github.com (<https://github.com>) (es gratis).

2.2 · Elige tu cliente

Hay dos formas igual de válidas de trabajar con Git en este ejercicio. El resultado en GitHub es idéntico — elige la que te resulte más cómoda:

Terminal o panel de VS Code

Escribes los comandos de `git` a mano, o usas el panel visual *Source Control* de VS Code. Más control, y es lo que verás en la mayoría de tutoriales.

GitHub Desktop

Cliente gráfico oficial de GitHub: commits, historial y *push* a golpe de clic, sin escribir un solo comando. Buena opción si es tu primera vez con control de versiones.

2.3 · Opción A — Git por terminal / VS Code

1 Comprueba que tienes Git instalado: `git --version` en la terminal.

2 Configura tu identidad (una sola vez por ordenador), con el mismo correo de tu cuenta de GitHub:

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu-correo@ejemplo.com"
```

3 El icono de la rama en la barra lateral izquierda de VS Code (o `Ctrl` + `Shift` + `G`) abre el panel *Source Control*: ahí puedes ver cambios, escribir el mensaje de commit y confirmar sin salir del editor, en vez de usar la terminal.

2.4 · Opción B — GitHub Desktop

1 Descárgalo de desktop.github.com (<https://desktop.github.com>) e instálalo.

2 Ábrelo e inicia sesión con tu cuenta de GitHub (*File* → *Options* → *Sign in*, o te lo pide al arrancar la primera vez). Desktop configura tu nombre y correo de Git automáticamente: no hace falta tocar `git config` a mano.

3 No necesitas instalar Git por separado — GitHub Desktop trae el suyo integrado.

2.5 · Crear el repositorio

El repositorio tiene que ser público. La Arena lee tu entrega a través de la API de GitHub sin que tú le des permiso a nadie — eso solo funciona con repositorios públicos. Uno privado no se podrá evaluar.

Opción A — terminal / VS Code

- 1 En GitHub, pulsa el + arriba a la derecha → *New repository*.
- 2 Ponle un nombre (por ejemplo `webi-m1`), déjalo en **Public**, y crea el repositorio vacío (sin README, sin `.gitignore`: los añadiremos nosotros).
- 3 Copia la carpeta de tu proyecto local en tu ordenador, y dentro de ella:

```
git init
git add .
git commit -m "Primer commit: estructura inicial"
git branch -M main
git remote add origin https://github.com/TU-USUARIO/webi-m1.git
git push -u origin main
```

Opción B — GitHub Desktop

- 1 Menú *File* → *New Repository...*, ponle nombre (por ejemplo `webi-m1`) y elige dónde crearlo en tu ordenador.
- 2 Trabaja en la carpeta que ha creado con tu editor de siempre (VS Code); Desktop detecta los cambios solo.
- 3 Pulsa «**Publish repository**» arriba. Se abre un diálogo con la casilla «**Keep this code private**» **marcada por defecto** — **desmárcala** antes de publicar, o el repositorio saldrá privado y la Arena no podrá leerlo.
- 4 Si ya tenías la carpeta creada de antes (por ejemplo porque empezaste desde la terminal), usa *File* → *Add Local Repository...* en vez de «**New Repository**» para engancharla a Desktop.

2.6 · Cómo organizar los archivos

Puedes usar un repositorio por entrega, o uno solo para toda la asignatura con una carpeta por misión — la Arena acepta las dos cosas, porque también se entregan subcarpetas. Dentro, la M1 exige **HTML, CSS y JS en archivos propios**:

```
webi-m1/
├─ index.html    ← solo estructura (nada de onclick="...")
├─ styles.css    ← todos los estilos
├─ app.js        ← toda la lógica, enlazado con defer
└─ README.md     ← con «Uso de IA» y «Autopsia» (apartado 5)
```

Si en algún momento tu proyecto genera carpetas que no son código tuyo (por ejemplo `node_modules/` en misiones posteriores), añade un `.gitignore` con su nombre para no subirlas: «repo limpio» es uno de los 30 puntos automáticos.

Comitea por fases, no de golpe al final. Otro de los 30 puntos automáticos exige **al menos 5 commits**. Si haces un commit al cerrar cada fase — escribiendo el resumen en el panel de GitHub Desktop o con `git commit`, tanto da — lo cumples sin pensarlo y además queda un historial que cuenta la historia de cómo construiste el ejercicio, útil para la propia defensa oral. En GitHub Desktop: escribe un resumen corto en el campo de abajo a la izquierda, pulsa **«Commit to main»** y luego **«Push origin»** arriba para subirlo.

Si entregas una subcarpeta, los commits se cuentan solo sobre ella. Un repositorio con 30 commits en el que la carpeta `m1/` se creó de una tacada no llega a los 5: comitea dentro de la carpeta que vas a entregar.

3 ENTRENAMIENTO

El oráculo de los números 🎱

Esto es el ensayo, no la entrega. Es la «Misión 1» del Nivel 2 del cuaderno de la U1: un ejercicio guiado, con el mismo código de partida para toda la clase, pensado para que aprendas a trabajar con la IA por fases. Lo que subes a la Arena lo construyes tú en el apartado 4.

Un oráculo digital ha pensado un número del 1 al 100 y desafía a quien visite tu página a adivinarlo. Tu trabajo: construir el templo (HTML) y la mente del oráculo (JS).

Objetivo

Construir el juego «Adivina el número» completo en el navegador.

Lo que practicas

`const` / `let`, `Number()` y comparación estricta, condicionales, template literals, DOM (`querySelector`, `textContent`), evento `click`.

Código de partida

Crea una carpeta para el entrenamiento (por ejemplo `oráculo/`) con estos dos archivos:

```
<!DOCTYPE html>
<html lang="es">
  <head>
    <meta charset="UTF-8" />
    <title>El oráculo de los números</title>
    <script src="app.js" defer></script>
  </head>
  <body>
    <h1>🔮 El oráculo de los números</h1>
    <p>He pensado un número del 1 al 100. ¿Te atreves?</p>
    <input id="intento" type="number" min="1" max="100" />
    <button id="probar">Consultar al oráculo</button>
    <p id="respuesta">El oráculo espera...</p>
    <p id="marcador"></p>
  </body>
</html>
```

```
// app.js – el oráculo elige su número secreto:
const secreto = Math.floor(Math.random() * 100) + 1;
console.log("(psst... el secreto es", secreto, "- quita esta línea al acabar");
```

Cómo trabajar con Gemini en las cinco fases: no le pidas «hazme el oráculo entero». Ve fase a fase, pídele que te explique cada propuesta antes de aceptar el diff, y prueba en el navegador al final de cada una. Si algo no lo entiendes del todo, pregúntale «¿por qué así y no de otra forma?» antes de seguir — esa pregunta es gratis ahora y de pago en la defensa oral.

Fase 1 · Conexión

Comprueba que `app.js` se carga y que el secreto se imprime en consola. Selecciona el input, el botón y los dos párrafos con `querySelector`, guardados en constantes.



PROMPT SUGERIDO

Tengo este `index.html` y este `app.js` de partida (los tienes abiertos en el editor).

Quiero declarar constantes que seleccionen `#intento`, `#probar`, `#respuesta` y `#marcador` con `querySelector`.

Propón el cambio y explícame, línea a línea, por qué usas `querySelector` y no otra forma de seleccionar el DOM.

✅ Verifica tú: abre `index.html` en el navegador (clic derecho → *Open with Live Server*, o simplemente doble clic en el archivo), abre la consola (`F12`) y confirma que ves el número secreto impreso.

¿No te aparece «Open with Live Server»? Es una extensión de VS Code, no viene instalada por defecto — instálala una vez:

- 1 Abre el panel de Extensiones (icono de piezas de puzzle en la barra lateral, o `Ctrl + Shift + X`).
- 2 Busca «Live Server» (de Ritwick Dey) e instálala.
- 3 Vuelve a `index.html`, clic derecho → *Open with Live Server* — o pulsa «Go Live» en la barra de estado, abajo a la derecha.

Con Live Server, la página se recarga sola cada vez que guardas un cambio: cómodo para ir probando cada fase. No es imprescindible — hacer doble clic en `index.html` para abrirlo directamente en el navegador funciona igual de bien para este ejercicio, solo que tendrás que recargar tú a mano.

Fase 2 · Primera consulta

Añade un `addEventListener("click", ...)` al botón. Lee el valor del input y conviértelo con `Number()`. Muestra en `#respuesta` el número recibido con un template literal.

PROMPT SUGERIDO

Añade un listener de click al botón `#probar`. Dentro, lee `intento.value`, conviértelo con `Number()` y muéstralo en `#respuesta` con un template literal, por ejemplo "Has dicho: 42".
Explícame por qué hace falta `Number()` aquí – ¿qué tipo de dato es `intento.value` tal cual?

La trampa clásica de esta fase: `input.value` siempre es un *string*. Si comparas `"50" > 42` funciona por pura casualidad (JavaScript convierte el string al compararlo con `>` / `<`), pero deja de funcionar en cuanto compares con `===` en la fase siguiente. Pídele a Gemini que te lo demuestre en la consola quitando el `Number()` a propósito, y vuelve a ponerlo tú mismo — es el error número uno de este ejercicio.

Fase 3 · El oráculo responde

Compara con `===`, `<` y `>`: muestra « Mi número es mayor», « Mi número es menor» o « ¡Correcto!». Si el input está vacío o fuera de 1–100, responde « Eso no es un número válido» sin gastar intento.

PROMPT SUGERIDO

Ahora compara el número convertido con "secreto" usando `===`, `<` y `>` para mostrar los tres mensajes de pista. Antes de comparar, valida que `intento.value` no esté vacío y que el número esté entre 1 y 100; si no, muestra el aviso y no sigas. Explícame por qué comprobas `value === ""` en vez de comprobar solo si el número da NaN.

✅ Verifica tú: con el secreto visible en consola, prueba un número mayor, uno menor, el acertado, el campo vacío y un número fuera de rango (como 500). Los cinco casos tienen que responder lo que toca.

Fase 4 · El marcador

Lleva la cuenta de intentos con `let intentos = 0;` y muéstrala en `#marcador`: «Intentos: 4». Al acertar, felicita indicando cuántos intentos se han usado.

PROMPT SUGERIDO

Añade una variable `intentos` que empiece en `0`, fuera del listener. Cada consulta válida (no los avisos de valor no válido) debe sumar 1 y actualizar `#marcador` con "Intentos: N". Al acertar, el mensaje de éxito debe incluir el número de intentos. ¿Por qué esta variable tiene que vivir fuera del listener y no dentro?

Fase 5 · Fin de partida

Cuando el jugador acierte, desactiva el botón con `boton.disabled = true;` para que la partida termine de verdad.

PROMPT SUGERIDO

Cuando el número acertado sea igual al secreto, además del mensaje de victoria, deshabilita el botón `#probar` para que no se pueda seguir jugando. Explícame qué pasaría si un usuario pulsa el botón varias veces seguidas sin este cambio.

Criterios de aceptación

- ☐ El script se carga con `defer` y no hay errores en la consola.
- ☐ Un valor vacío o fuera de rango muestra aviso y no incrementa los intentos.
- ☐ Las pistas mayor/menor son correctas en todos los casos (pruébalo con el secreto visible).
- ☐ El contador de intentos se actualiza en pantalla en cada consulta válida.
- ☐ Al acertar, el mensaje incluye el número de intentos y el botón queda desactivado.
- ☐ No queda ningún `console.log` con el secreto en la versión final.

Retos extra (opcionales)

- Límite de **7 intentos**: si se agotan, el oráculo revela el número y bloquea el botón (logro «Francotirador»).
- Un botón «Nueva profecía» que reinicie el juego (nuevo secreto, contador a 0, botón activo).

- Guarda cada intento en un array y muéstralos todos: «Has probado: 50, 75, 62».

Logro 🧠 **Sin red**. Si quieres el reto de verdad, intenta cada fase sola contigo antes de pedirle a Gemini que la resuelva, y solo pídele ayuda cuando te atasques de verdad. Se nota — y se defiende mejor — el código que primero intentaste tú.

4 LA ENTREGA

M1 · El Despertar del DOM ✂

Sin frameworks. Sin librerías. Tú contra el navegador.

Esta es la misión que se entrega y se puntúa. El enunciado es abierto a propósito: una página interactiva en **HTML + CSS + JavaScript puro** — un mini-juego, un generador, un tablero, lo que se te ocurra — con todo el DOM manipulado a mano. **Prohibido React, prohibido jQuery.**

4.1 · Cómo se puntúa (sobre 100)

30 · Comprobaciones automáticas

README con sus secciones, repositorio limpio, **≥ 5 commits**, tecnología correcta (nada de frameworks) y estructura de archivos.

70 · Rúbrica evaluada por IA

Sobre tu código concreto, con feedback por criterio y **3 preguntas de defensa** que solo ve el profesorado.

CRITERIO	QUÉ MIRA	PUNTOS
Manipulación del DOM	Selección, creación y modificación de nodos de forma correcta y eficiente	20
Eventos	Manejo bien estructurado, sin handlers inline en el HTML	15
Fundamentos JS	Tipos, ámbitos, funciones y template literals usados con criterio	15
Calidad y organización	Código legible, nombres claros, sin duplicación evidente	10
Originalidad	La idea y la ejecución van más allá del mínimo	10

 **Bonus de la misión:** un **modo oscuro que se active con una tecla secreta**. Es el reto que propone la M1 — pista: el evento `keydown` sobre `document` y `classList.toggle` sobre `<body>`.

4.2 · Elige qué construir

No entregues el oráculo tal cual. Lo hemos hecho todos con el mismo código de partida: como entrega se lleva un cero en **Originalidad** (10 puntos) y le deja muy poco que decir al evaluador en «Calidad». Además, tres preguntas de defensa sobre un código que es idéntico al de tu compañero no son una buena tarde para nadie.

- **El reto jefe del cuaderno, «Caza al Bug»** (Nivel 3): un *whack-a-mole* de 3 × 3 con marcador, temporizador y fin de partida. Es el candidato natural: si lo haces, ya tienes la M1 casi lista — añádele CSS propio, tu vuelta de tuerca y el README.
- **Tu propia idea.** Un memory de cartas, un generador de personajes con tiradas de dados, un tablero de tareas con arrastrar y soltar, un simulador de máquina expendedora, un teclado musical, un contador de calorías, un buscaminas pequeño... Vale cualquier cosa que reaccione al usuario.
- **El oráculo irreconocible.** Si te has enamorado del ejercicio, conviértelo en otra cosa: modo por rondas contra el reloj, un rival que también adivina, pistas de «caliente/frío» con color, historial visual de intentos, dificultad seleccionable. Con idea propia, CSS propio y el bonus, deja de ser el ejercicio de clase.

4.3 · Lo que hunde la nota (y es facilísimo de evitar)

- `onClick=""` en el HTML: los eventos se enganchan desde el JS con `addEventListener`. Es el criterio de Eventos, 15 puntos.
- `var`: usa `const` por defecto y `let` solo cuando el valor cambie.
- `innerHTML` con texto que escribe el usuario: usa `textContent`. Lo primero abre la puerta a inyectar HTML; lo segundo no.
- Todo en un único archivo, o estilos dentro del `<head>`: HTML, CSS y JS separados.
- `console.log` de depuración olvidados y código muerto comentado.
- Un solo commit gigante al final: son 5 mínimo, y se ven.

4.4 · Trabajar tu proyecto con Gemini

El mismo método del apartado 3, ahora sobre una idea tuya: primero el plan, luego fase a fase, y una revisión crítica al final.

PROMPT SUGERIDO

Voy a construir [describe tu idea en dos frases] con HTML, CSS y JavaScript puro, sin frameworks ni librerías. Antes de escribir código: propón 4 o 5 fases pequeñas para construirlo, cada una con algo visible funcionando al terminarla, y dime qué estado necesito guardar en JavaScript y por qué. No escribas todavía ningún archivo.

PROMPT SUGERIDO

Revisa mi index.html, styles.css y app.js como si fueras el profesor que va a corregirlos con esta rúbrica: DOM 20, eventos 15, fundamentos JS 15, calidad 10, originalidad 10. Dime los tres puntos más débiles y por qué, sin arreglarlos todavía – quiero intentarlo yo primero.

Y el prompt más rentable de todos, antes de entregar: «Hazme tres preguntas difíciles sobre *mi* código, del estilo de una defensa oral, y no me des las respuestas». Si alguna te deja en blanco, ahí tienes lo que te falta por entender.

5 DOCUMENTACIÓN

El README: cómo declarar el uso de IA

Toda entrega a la Arena necesita un `README.md` con dos secciones obligatorias. No puntúan por cuánta IA usaste — puntúa que lo declares con honestidad y lo puedas defender.

SECCIÓN	QUÉ DEBE CONTENER
Uso de IA	Qué partes hiciste con IA y con qué herramienta, 1–2 prompts reales relevantes, cómo verificaste lo generado y qué escribiste a mano. Si no usaste IA, dilo y listo.
Autopsia	Las 2 decisiones más discutibles de tu código y qué alternativa descartaste en cada una.

Plantilla

Caza al Bug

Misión M1 · El Despertar del DOM – Web Development I.

Cómo probarlo

Abre index.html en el navegador (o con Live Server). Pulsa «Jugar»: tienes 30 segundos para aplastar bugs. Tecla secreta: pulsa "n" para el modo nocturno.

Uso de IA

Usé Gemini CLI (VS Code) como pareja de programación, fase a fase. Ejemplo de prompt real: "Propón 4 o 5 fases pequeñas para construir un whack-a-mole en JS puro; no escribas código todavía". Verifiqué cada cambio jugando una partida completa antes de aceptar la siguiente fase, y probé los casos límite (clic en casilla vacía, clic después del fin de partida). Escribí a mano: el setTimeout encadenado que hace aparecer los bugs, que la primera propuesta resolvía con setInterval y no se detenía bien al acabar la partida.

Autopsia

1. Guardo la posición del bug activo en una variable del módulo en vez de leerla del DOM cada vez, porque el DOM solo debería reflejar el estado, no ser la fuente de verdad. Descarté buscar la casilla por su clase CSS: es más frágil y acopla la lógica al aspecto.
2. Uso un solo listener en el contenedor de la cuadrícula (delegación) en vez de nueve listeners, uno por casilla. Descarté los nueve porque al reiniciar la partida habría que quitarlos y volver a ponerlos, y es justo donde aparecen los bugs de eventos duplicados.

Escríbelo tú. Es el único documento de la entrega que la IA no puede rellenar por ti sin que se note: habla de decisiones que tomaste, no de lo que hace el código.

6 ENTREGA

Subir a GitHub y entregar en la Arena

6.1 · La primera vez que entras en la Arena

- 1 Escribe tu correo de U-tad en **/login**. No hay contraseñas: recibes un enlace de acceso por correo.
- 2 Abre el enlace y **pulsa el botón** de la página (el enlace no abre sesión solo).
- 3 Elige tu **alias**: es con el que apareces en el ranking, y se elige **una sola vez**.

- 4 Elige tu **clase** (castellano o inglés). También una sola vez: cambiarla después es cosa del profesorado.

6.2 · Entregar

- 1 Comprueba que el `README.md` con sus dos secciones está en la carpeta que vas a entregar.

- 2 Guarda el trabajo con commits, uno por cada fase que hayas ido cerrando, y súbelo:

```
git add .
git commit -m "Fase 3: fin de partida y marcador"
git push
```

- 3 Comprueba que el repositorio es público: ábrelo en una ventana de incógnito — si lo ves sin haber iniciado sesión, está bien.

- 4 Entra en la Arena → **/entregar**.

- 5 Pega la URL de tu repositorio (vale la del repo entero, o la de una subcarpeta si la misión vive dentro de un repositorio con varias, tipo `.../tree/main/m1`).

- 6 Elige **M1 · El Despertar del DOM** en el desplegable y confirma.

Qué pasa después: tu entrega queda en estado «pendiente» hasta que el profesorado lance la evaluación. Cuando termine verás tu nota y todo el feedback por criterio. Las 3 preguntas de defensa se generan sobre tu código pero **no** se te enseñan: la defensa mide si entiendes lo que entregaste, no si te has preparado tres respuestas. **Tu nota es provisional hasta esa defensa.**

Puedes reentregar. Corriges, haces push y vuelves a entregar la misma URL: en el ranking cuenta **el mejor intento** de cada misión. Eso sí, hay un límite de **5 entregas cada 10 minutos** — no es un botón de recargar.

✅ ANTES DE ENTREGAR

Checklist final

- ☐ Gemini CLI instalado, con la API key en la variable de entorno y enganchado a VS Code.
- ☐ La API key **no** está en ningún archivo del repositorio.
- ☐ Repositorio en GitHub creado como **público** (comprobado en incógnito).

- ☐ Al menos 5 commits **dentro de lo que entregas**, repartidos por fases.
- ☐ HTML, CSS y JS en archivos propios; ni un `onclick=""` en el HTML.
- ☐ La idea es **tuya**, no el oráculo de clase tal cual.
- ☐ Ningún `console.log` de depuración ni código muerto.
- ☐ README con las secciones «Uso de IA» y «Autopsia» rellenas de verdad.
- ☐ Puedes explicar cada línea de tu código sin mirar el prompt que la generó.
- ☐ URL entregada en /entregar, con la misión «M1 · El Despertar del DOM» seleccionada.

Y lo único que de verdad importa: si puedes contar tu código en voz alta, sin mirarlo, a alguien que no lo ha visto — ya has aprobado la defensa.